Mission scoring – Pilotwings 64 (USA)

Findings for the planned Mission tab of Hawk's Hangar. Analysis of the decompilation and the retail ROM, as of 20 October 2026 (eighteenth revision: stage 5 – game patch "Bonus games and Birdman": result tables [7][4][5] in the data area, texts per level for Sky Diving and the Jumble Hopper, Birdman medal limits and stage 4 in the save file; seventeenth revision: stage 4 – time points after a crash, the Gyrocopter's touchdown curve, good landing values per mission; sixteenth revision: hard end jingle, Jumble Hopper without landing accuracy; fifteenth revision: stage 3 – data area, game patch "goal objects", goal time points, hard end time per mission, flags of vehicles that end in the air).

Terms used in this document

Term	Meaning
Mission (= test)	one single task, e.g. Gyrocopter Beginner "Novice Rings". One UPWT file each.
Class	the group of up to five missions of one vehicle: Beginner, A, B, Pilot; for the bonus games the levels, for Birdman the stages. Retail uses 1–4 missions per class; the game's tables allow 5.
Vehicle	Hang Glider, Rocket Belt, Gyrocopter, Cannonball, Sky Diving, Jumble Hopper, Birdman.
Source	one kind of points (a curve or an object type), shown as one row of the score sheet.

"128 missions" therefore means 128 single tasks in total, across all vehicles and classes (retail: 61).

Names: the game's "Meca Hawk" is called *falco* in the decompilation.

1. Where the data lives

- Each test is one UPWT file. File groups: 005–016 Cannonball, 017–019 Sky Diving, 020–022 Jumble Hopper, 023–038 Birdman, 039–047 Gyrocopter, 048–056 Rocket Belt, 057–065 Hang Glider.
- COMM chunk (1072 bytes in every mission): byte 0 = class (0 Beginner, 1 Class A, 2 Class B, 3 Pilot; bonus games and Birdman: level/stage), byte 1 = vehicle, byte 2 = test, byte 4 = condition mode (section 4). The scoring struct `Unk80345C80` starts at COMM+0x48.
- JPTX chunk: text base name of the test (e.g. `E_GC_1` → `_N` title, `_M` task, `_H` hint). Bonus games: `A_EX_n` Cannonball, `B_EX_n` Sky Diving, `P_EX_n` Jumble Hopper (n = level).
- Game menu order: Hang Glider, Rocket Belt, Gyrocopter; classes Beginner (1 test), A (2), B (3), Pilot (3). Bonus: Cannonball 3 levels × 4 targets, Sky Diving and Jumble Hopper 3 levels × 1 test. Birdman 4 stages × 4 missions.

1.1 Photo curves and the rest of the COMM block (corrected in stage 1)

- COMM+0x48+0x1A4 to +0x39C holds **six photo curves** (not free space, as the earlier analysis assumed): when a photo is taken, snapPhoto (snap.c) evaluates one of them per subject – ferry, fountain, missile, flame, shuttle, whale – with the subject's animation timing as input. The three photo missions of the Hang Glider fill them; Cannonball level 1 target 1 has a leftover of 25 points in every slot; all other missions have "(0, 0)".
- COMM+0x414 (f32): Jumble Hopper deduction for leaving the area (2 in all missions); COMM+0x418 is 1.0 in the Jumble Hopper missions, use unknown.
- Really unused are only bytes 5–7 and the 4 bytes behind the counts (0x42C–0x42F).
- **Consequence (built in stage 3):** new per-mission settings go into the **mission settings table** in the editor's data area (section 13.4), one entry of 0x100 bytes per mission with settings. The game module reads the entry of the mission being flown; the editor writes the table when building.

2. How a score is put together (`func_8032B560`, code_B2900.c)

2.1 Curves

Five curves `{s32 count; Vec2F points[8]}` (0x54 bytes each), evaluated piecewise linearly (`func_80313F08`). The highest y value of a curve is the maximum for that source.

Offset	Flag bit	Input	Result row
+0x000	0x2 / 0x1	distance to the landing point	Landing Accuracy (type 2)
+0x054	0x8	distance travelled	never shown (unused in retail)
+0x0A8	0x10	maximum altitude	Altitude Pts. (type 6)
+0x0FC	0x800	elapsed time	Time Pts. (type 3)
+0x150	0x4	landing impact (value from the vehicle code)	Landing Impact (type 1)

All flag bits of the result (`arg0->unk0`): 0x1/0x2 landing accuracy, 0x4 impact, 0x8 distance, 0x10 altitude, 0x20 rings, 0x80 photos, 0x100 targets, 0x200 balloons, 0x400 deductions, 0x800 time, 0x1000 Sky Diving formations, 0x2000 Cannonball, 0x4000 hover pads, 0x8000 Meca Hawks, 0x10000 ball targets, 0x20000 hopper goals.

2.1a Height units

- Positions (mission files, RAM) are in game units. The altimeter shows the height **above the ground × 0.7** (`hud->altitude = height * 0.7f` in every vehicle). The original texts use the same meters: HG Class B 1 says "ascend to 400m", its mode-4 value is 571 (× 0.7 = 399.7).
- The highest altitude of the result (altitude curve, mode 4, module condition) is the vehicle's **z above sea level** (shown on the HUD under "SEA LEVEL", also × 0.7) (game.c func_802EDAF0), not the height above the ground. A start on a tower or mountain

already counts (HG Beginner 70 units = 49 m, RB Beginner's balloon hangs at z 182 = 127 m, so popping it nearly reaches an altitude condition of 180 units).

- The editor shows and takes heights in the game's meters (units × 0.7) and stores units (`GameUnits`); the landing accuracy curve (distances) likewise.

2.2 Landing accuracy and the Gyrocopter

- All vehicles: accuracy = curve(distance between the **resting position** and the nearest landing pad centre or runway). The distance is measured every frame (`pads.c`); the last value counts.
- Gyrocopter only: it also stores the distance at the **first wheel contact** (touchdown point, `landedDist`). Then accuracy = curve(resting position) × curve(touchdown point). Both use the same curve, so its maximum is the square root of the shown points (5.477 → 30, 4.472 → 20). Landing on the runway and rolling out of the target area costs points; touching down outside and rolling in does too.

2.3 Landing geometry

- **Landing pad (circle):** the input is the horizontal distance from the pad centre. The curve turns it into points, so a mission has a "perfect" circle (plateau, e.g. 6 m) and a circle from which there are no points (last x, e.g. 35 m), with linear steps in between (retail: 2–4 intermediate points, e.g. 6 m → 40, 11 m → 30, 18 m → 20, 35 m → 4, then 0). A good landing needs the resting position inside the pad radius (30 units = 21 m, code; per mission since stage 4), so for Hang Glider and Gyrocopter the curve beyond 21 m never counts.
- **Runway (LSTP: two end points + alignment width):** inside the circle around the runway centre with radius half the runway length, and between the two end lines, the input is the **distance from the centre line** (3D, so height counts a little). Along the runway there is no falloff at all. Outside, the input is the distance to the runway centre (⩾ half the length, normally 0 points). A good landing on a runway: distance from the centre line below half the alignment width.
- So it is not a rectangle with falloff along two axes: on a runway only the sideways offset counts. The Gyrocopter multiplies the sideways offset at touchdown with the one at rest.
- With more than one pad or runway, the nearest one counts.

2.4 Landing impact (sentence per vehicle for the editor)

Vehicle	How the impact value is measured
Hang Glider	vertical speed at touchdown (× 2.52); a crash at 15 or more (vertical or forward)
Rocket Belt	speed into the ground at touchdown; above 7.93 a hard landing, above 21.8 a crash
Gyrocopter	highest speed of a wheel into the ground at the first touchdown; crash at 12.698, off a runway or pad at 7.937
Sky Diving	vertical speed at touchdown
Jumble Hopper, Birdman, Cannonball	not measured (always 0, so an impact curve would always give its maximum)

2.5 Object points

Objects score through their own data in the mission (COMM object chunks):

Source	Where the points are	Row type
Rings	per ring type: COMM+0x39C s16[4]; timed rings: ring.points	4
Balloons	BALS.points per balloon	5
Targets (missiles)	per target	7
Meca Hawks	per Meca Hawk	8 "Destroyed"
Hover pads ("Floating Pads")	HPAD.points per pad	9
Photos	computed from the photo (PHTS)	11
Formations	SDFM points per formation	16
Ball target, hopper goal	per object	never shown (see 3.2)

Linked rings: of a group of child rings only one counts; the others vanish. So a plain sum of all ring values is not the reachable maximum.

2.6 Sum

All sources + deductions (COMM+0x3B4 per hit, GC −5, RB −2, at most −100) are added, then clamped to 0–100. Sources are only counted if their flag bit is set (section 3).

2.7 Other fields

Offset	Meaning
+0x39C s16[4]	ring points per ring type (type 2 = invisible check ring, always 0; type 3 = goal ring)
+0x3A4 f32[4]	extra ring value (rarely used)
+0x3B4 s32	deduction per hit
+0x3B8 s32	score threshold (70) for the pilot's good/bad reaction
+0x3BC f32	"Land now" time in s (0 = off)
+0x3C0 f32	hard end time in s (0 = off)
+0x3C4 f32	required object count (mode 3)
+0x3C8 f32	altitude threshold (mode 4)

2.8 Distance points (built)

- The distance curve (COMM+0x54, flag 0x8) takes the distance flown (the whole path, result unk14 = player unkC, summed per frame). Every vehicle except Cannonball sets flag 0x8, so it counts already; it is only missing on the result sheet: resultInit picks each row's points through a jump table (0x80355CA8) without a case for row type 10. The editor offers "Distance" as a row (type 10) and, when used, points that table entry to a two-instruction stub in the game module (`j 0x8032E528; lh a1, 8(t0)`). Checked in the emulator.
- Time condition "distance flown at least N" (module only, combinable with all others): entry flag 4, f32 at +0x10 of the mission's entry (section 13.4).

2.9 Start position (TPAD)

- The start is the mission's TPAD chunk (TaskTPAD, 0x30 bytes: position, angles in degrees, a vector at +0x1C, a byte at +0x28, the Rocket Belt's fuel at +0x2C), read by db_getstart. It is no object of the island; without a TPAD the game starts in the middle of the terrain box.
- Fuel (built): Rocket Belt (data +0xF8) and Gyrocopter (+0xD8) start with TPAD+0x2C (0 to 1; retail 1, RB A2 0.25, RB P3 0.2). The editor writes a changed start fuel into the mission file at build. "Unlimited fuel" is an entry flag (8): the module refills the tank after every flight frame (func_802EDAF0 hook), and the two calls of hudDrawFuel (gauge and "FUEL" label, in hudDrawRocketPack and hudDrawGyrocopter) go to the module, which skips them. An entry with only this flag leaves the game's own condition byte in charge. Checked in the emulator.

3. Flags: what counts when

3.1 From the vehicle code (end of the flight)

Vehicle	After a good landing	Otherwise	During the flight
Hang Glider	0xFFFF (all)	0xF7F8; touching down against an obstacle on a pad 0xF7FB	**0** (set only on landing or crash)
Gyrocopter	0xFFFF	0xF7F8	0xF7F8 (set at the start)
Rocket Belt, Sky Diving, Jumble Hopper	0xFFFF from the start (never changed)	–	0xFFFF
Cannonball	0x0002 (landing accuracy only)	–	–
Birdman	0xF7FB (accuracy, no impact, no time)	0xF7F8	**0** (like the Hang Glider)

0xF7F8 means no landing accuracy, no impact and no time points, but all object points. If the vehicle hardly moved (altitude and distance below 2), all flags are cleared.

Found in stage 3 (emulator): a Hang Glider that ends in the air (goal ring) has flags 0, so it would score nothing at all, not even its rings. The game module therefore gives a vehicle whose flight ends at the goal with flags 0 the flags 0xF7F8 (section 7).

3.2 Overrides in the scoring function (the same for all vehicles)

- **Crash:** flags = 0x3F7F8 → object points, deductions, altitude and distance count; landing, impact and time do not.
- **Hard end time reached:** flags = 0x3F3E0 → only object points (not even the deductions, altitude or distance). Replaced per mission by the module (section 4.3).
- The flags are passed as 16 bits from the vehicles, so ball target (0x10000) and hopper goal (0x20000) only ever count after a crash or the hard end. That is why hopper goal points are always 0 in a normal run. Fixed by the game patch "goal objects" (section 9.1).

Both overrides are constants in **one function**, before the call of taskGet_80346364 (ROM 0xB2AE4). Since stage 3 that call always goes to the game module (`module_conditions`), which adjusts the flags per mission and then returns the condition mode.

3.3 Points after a crash (built, stage 4, no patch button)

Built (Step 34): the Missions tab offers three choices under "After a crash" (not for Cannonball; Birdman follows in stage 5): as in the original game, the time points count too, or 0 points.

- "0 points" is the old check box (`MissionConditions.CrashGivesZero`; alone the game's mode 1).
- "Time points too" (`MissionConditions.CrashTimePoints`, entry flag 64, module only): `module_conditions` adds the time bit (0x800) after a crash and writes the **time of the**

crash into the result (+0x1C). The module records it in its flight frame hook: the vehicle's flight time (argument 6 of func_802EDAF0) in the first frame of the **impact** (flight state, argument 10, = 3). After an impact every vehicle tumbles for 3.75 seconds (HG unk8C 2, RB unk90 2, GC unkC0 2, BM unk104 1) before the game marks the crash (state 4, result +0x3C); the game itself would use the last time the wreck still moved. The conditions of the time points apply afterwards. Checked in the VM and in the emulator (HG Beginner "Conditions: crash at 20.3 s, time points count"; the crash flag came at 24 s).

- From the impact on the hard end countdown stops (no numbers, no "Time up!", no sounds). If the hard end time is reached while the vehicle tumbles, the module scores the flight as a crash (flags 0x3F7F8, crash flag set) and plays no hard end jingle; the original would score it like a hard end. Checked in the emulator (hard end 22 s, impact 20.3 s: "5" … "2", then nothing; scored as a crash). The flight frame hook is built in for every mission with a hard end setting for this.
- The Missions tab warns when the time curve gives points at 0 s ("a crash right after the start gives N time points"). The automatic footnote says "Time points also count after a crash." (short form "Crash: time pts. count.").

Original plan: The per-mission crash mask decides which sources still count after a crash:

- **Retail** (default when the patch is applied): objects, deductions, altitude, distance.
- **Time points after a crash** (new option): the mask also gets the time bit (0x800). The time curve then gets the time of the crash as input – with a decaying curve, an early crash gives many time points. The mode conditions (all objects, N objects, altitude) still apply.
- **Nothing** (like mode 1 "crash → 0", but combinable with modes 2–4).
- The conditions per object type (4.4) apply to the time points after a crash as well, e.g. "time points after a crash only if all rings are cleared" or "… at least 5 balloons".
- Landing and impact after a crash are possible in principle (same mask), but make little sense; the editor does not offer them.

4. Conditions and time limit

4.1 Mode byte (COMM+4)

Mode	Effect
0	no condition (objects still count after a crash)
1	crash → 0 points in total
2	time points only if all objects are cleared
3	time points only if at least COMM+0x3C4 objects are cleared ("land now" when reached)
4	time points only if the maximum altitude ≥ COMM+0x3C8

The modes exclude each other: "crash → 0" cannot be combined with modes 2–4. A per-

mission crash setting in the mission settings table (sections 1.1 and 3.2) removes this limit.

4.2 Time curve

- Usual shape, plateau + linear decay, e.g. GC Beginner: (0, 30) (90.5, 30) (120, 15) (210, 0). The text says "deducted when 1'30"": the half second is grace, and the text time is the second x value.
- "As close as possible" shape (HG Class B 3): (0, 0) (150, 0) (150, 40) (177, 70) (183, 70) (210, 40) (270, 0), i.e. "close to 3'00"".

4.3 Time events

- **"Land now"** (COMM+0x3BC): after this time the HUD shows "Land now" and the mission objects are removed. Retail: HG B3 180 s, HG P1 240 s.
- **Hard end** (COMM+0x3C0, `D_80362690->unkA1` is set when it is reached): after this time the game goes straight to the results, also in the air. Unused in retail; works (checked in the emulator: HG Beginner with 20 s ends at 20 s with the result sheet).
- **Built (stage 3), no patch button:** the Missions tab edits the time ("Hard end after N seconds", 0 = off; not for Cannonball) and a choice per mission (`MissionSettings.HardEndZero`):
  - **"The flight is scored"** (default): flags = 0x3FFF8 – everything but landing accuracy and impact (objects, deductions, altitude, distance, time with the end time as input).
  - **"The flight gives 0 points"**.
  - The retail rule (only objects) is not offered.
  - A crash keeps the crash rules. The conditions of the time points still apply afterwards.
  - Every mission with a hard end time gets an entry in the mission settings table (flag 16 = scored, 32 = 0 points); the module applies it in `module_conditions`.
  - The automatic footnote says "The flight ends after 3'00"." or "… with 0 points." (short forms "Ends at 3'00"." / "Ends at 3'00": 0 pts.").
  - Intended for Birdman missions without a landing and as a strict time limit.
  - **Jingle (built 2026-10-16):** the game plays nothing at the hard end itself; only the Hang Glider's result frames (func_802EEB00) play its landing jingle. Per mission the editor offers "landed successfully", "landed wrongly" or "no jingle" (`MissionSettings.HardEndJingle`; default "successfully" when scored, "wrongly" with 0 points). Music numbers: HG 0x05/0x06, RB 0x09/0x0A, GC 0x0D/0x0E, SD 0x14/0x15, JH 0x18 (goal) / 0x0A (the Rocket Belt's, JH has no "missed"), BM 0x1A / 0x1B (crash, BM has no "missed"). Entry byte +0x14. "arg0->unkA1 = 1" (ROM 0x75004) goes to `module_hard_end`, which plays the jingle at once; the Hang Glider's sndSetMusic/sndSetMusicState calls (ROM 0x760D4, 0x76118, 0x76128, 0x760DC, 0x76130) go to the module, which skips them after a hard end with a setting. Checked in the emulator (HG none/missed, RB OK).
  - **Countdown (built 2026-10-16):** in the last 5 seconds a red "5" … "1" (bold, font 6, scale 2) in the middle of the picture (y 70), then "Time up!" until the result sheet.

hudMainRender calls hudDrawStartText through `module_hud_text` (ROM 0x9FBD4), which then draws with the game's own uvFontPrintStr16 (visible in every emulator). Remaining time = hard end (COMM+0x3C0) − flight clock D_8034F850. The texts come from the editor (module header +0xF0, 6 × 12 codes). Checked in the emulator ("3" at 17.7 s, "1" at 19.0 s, "Time up!" at 20.0 s). Sounds: each number plays SFX 0x43 (the photo reticle appearing when Z is held), "Time up!" the goal gong SFX 0x11 (Jumble Hopper goal); each once, when the text changes.

4.4 Conditions per object type (built, through the game module)

Built (Step 29): the Missions tab has check boxes: "a crash gives 0 points" (not for Cannonball) and, for the time points, a count per object type present in the mission ("all" or "at least N") plus a minimum altitude, all combinable. Whatever the retail mode byte can express (one condition alone, "all objects" for exactly the types present, "N objects" of the only countable type) is written there, so the HUD helpers ("n left", "land now", HG altitude mark) keep working. Combinations go into the mission settings table (data area, section 13.4, up to 128 missions); the scoring function calls `module_conditions` (ROM 0xB2AE4) instead of `taskGet_80346364`, which applies the conditions and returns 0, or the retail mode for missions without conditions. Balloons are counted by the module itself (popped entries of `gBalls`). Checked in the emulator (all seven counters, crash → 0; again after the move into the data area). HUD counter (built): the ring and target code ask `func_8032C080` how many objects are still needed (ROM 0xAB55C in rings_80323FFC, 0xCBD10 in targets_8034473C) and show one less as "n to go"; retail answers only in mode 3. With module conditions both calls go to the module, which answers with what the mission's ring or target condition still needs and never reports "reached" (no "please land"). Checked in the emulator ("Ring cleared, 1 to go"). Balloons, hover pads and the other types have no such message in the game.

Original plan: Today only one condition per mission is possible (mode 2: all objects of all types, mode 3: N objects of all types together, mode 4: altitude). Planned instead:

- Per mission, for every object type a required count: rings, balloons, targets, Meca Hawks, hover pads, ball targets, hopper goals – each "not required", "all" or a number.
- Time points only count if **every** condition is met. This applies to the normal end, the goal ring and the new "time points after a crash" option (3.3) alike.
- The values live in the mission settings table (1.1). A new small routine (in the free code area behind 0x803805E0) replaces the two retail check functions; it also feeds the "n left" counter on the HUD and the "land now" message.
- Retail detail: the mode-3 count leaves out balloons (there is no counting function for popped balloons, only "all popped yes/no"). The new routine counts them itself.
- The modes 2–4 can be expressed with the new conditions (mode 4 altitude stays as it is), so the editor converts them when the patch is applied, and back on undo where possible (otherwise it lists the missions that lose a condition).
- Effort: medium (new code instead of a few changed instructions).

4.5 HUD "Time requirement met" (built)

When the object that completes all time conditions is cleared (rings, targets), the module shows "Time requirement met" as the second HUD line: it calls hudWarningText with the "to go" text and overwrites the copy (sHudState+0xB40) with the editor's codes (in-RAM form: line break 0xFFE, end 0xFFFF). "0 to go" is never shown.

4.6 "View Demo" button (built)

The hint screen of Beginner HG/RB/GC shows "View Demo" (test_menu.c, three checks "class Beginner and vehicle < 3"). Per vehicle it can be hidden: each check becomes `lui at, mask; sllv at, at, veh; bgez at` (mask bit 31 − vehicle = keep the button).

5. Good landing

5.1 What decides it today

Vehicle	Good landing	Values (code constants)
all with pads	resting position within the pad radius	radius 30 game units = **21 game meters** for every pad (pads.c, set when a mission loads)
all with runways	within the runway rectangle and half its alignment width	from the mission (LSTP +0x20, per runway)
Hang Glider	touchdown on the ground with "flare" (unk88 > 0.75) and both speeds below 50 km/h = landing, else crash; good landing below 15 km/h vertical and forward on a pad	50, 50, 15, 15 (immediates in hangGlider_802F3A80; its speeds are already × 2.52 = km/h)
Gyrocopter	stop on a pad/runway; crash at a wheel speed into the ground ⩾ 12.698 units/s (32 km/h), beside a pad or runway ⩾ 7.937 (20 km/h)	constants ROM 0xDACF8, 0xDACFC (func_80305DC0)
Rocket Belt	on a pad/runway ("Landing OK"); crash above 21.8 units/s into the ground (54.9 km/h), hard landing (impact flag cleared) above 7.93 (20 km/h) into the ground or sideways	constants ROM 0xDC964, 0xDC96C (crash), 0xDC974 (hard), 0xDC978, 0xDC988 (hard sideways); 0xDC934/38 are the demo's
Sky Diving	on a pad: the impact flag (4) is set (correction: there is no own radius of 25; Sky Diving uses the pad radius too)	–
Jumble Hopper	reaching the goal	–

The HUD shows speeds as units per second × 3.6 × 0.7 (= × 2.52) km/h; the editor shows all these speeds in km/h and the radius and runway widths in game meters.

5.2a Built (Step 34, no patch button)

- Missions tab, section "Good landing" (only the values of the vehicle shown): pad radius (if the mission has landing pads; not Cannonball, Jumble Hopper), runway widths (written into the mission file, `MissionSettings.RunwayWidths`), and the speeds of the table above (`MissionSettings.Landing`, stored form; only values that differ from the original).
- Entry: +0x15 landing flags (1 pad radius, 2 speed limits), +0x6C f32 radius, +0x70 f32[4] limits in the order of `LandingValues.SpeedsOf` (HG landing vertical, landing forward, good vertical, good forward; GC crash, crash beside; RB crash, hard, hard sideways). Module header flag 128 when any mission has them.
- The flight frame hook sets them in every frame (or the originals for other missions, the title demo included): the radius of every used pad (gLandingPads 0x8036C2E8, +0x10), the RAM copies of the GC/RB constants, and four f32 at **0x801F5040** (data area header +0x40) that the patched Hang Glider code loads: at ROM 0x7B1F8/0x7B21C/0x7B2D0/0x7B2D8 `lui at, 0x4248/0x4170` → `lui at, 0x801F`, and the `mtc1 at, fN` using them (0x7B208, 0x7B218, 0x7B230, 0x7B2D4, 0x7B2EC) → `lwc1 fN, 0x5040…0x504C(at)`. The GC/RB constants are only checked against the ROM (no code change).
- Checked: VM (values per mission and back to the originals), emulator RAM during an HG flight (the patched words, 1.0/1.0/15/15 for a mission with changed limits). **Not yet checked: a real landing with changed values** (the scripted Hang Glider flights all crashed into the terrain).

5.2 Per-mission landing settings (patch "good landing", original plan)

- The values go into the mission settings table (1.1), one entry per mission: pad radius, runway width factor, Hang Glider crash speeds, Gyrocopter crash speeds, Rocket Belt hard-landing/crash speeds, Sky Diving radius.
- A small hook at mission start copies them over the constants (or, where a constant sits in the instruction itself, the instruction is replaced by a load).
- Effort: medium. Each vehicle needs its own small patch; every constant has to be located in the ROM first.

5.3a Built (Step 34, no patch button)

- Check box "Own curve for the touchdown point" (Gyrocopter, landing accuracy row). On: a copy of the landing curve becomes the touchdown curve (`MissionSettings.TouchdownCurve`, game form); both are shown in percent of their highest value, one point of each must be 100 %; the row's points (the product of both maxima) scale both curves by the same factor. Off: back to one curve.
- The scoring function's call of func_80313F08 for landedDist (ROM 0xB2C88) goes to `module_touchdown_curve` (only built in when a mission uses it): a stub that keeps a1 (the

distance, soft-float module) and jumps to func_80313F08 with the entry's curve (+0x18, entry flag 128) or the original one. Checked in the VM.

5.3 Separate Gyrocopter measurements (patch "two landing curves", original plan)

- Today touchdown point and resting position share one curve (2.2).
- Patch: the touchdown measurement reads a second curve from the mission settings table (1.1) instead of the shared one. A few changed instructions in the scoring function.
- The editor still shows one "Landing Accuracy" value. The two curves are shaped separately (perfect and zero distances); their maxima multiply to the shown value, so each gets √value by default.

5.4 Patch handling (all patches with "apply" and "undo" buttons)

- **Decision (2026-10-20):** stage 4 has no game patches: crash scoring, the touchdown curve and the landing values are settings of a mission, applied by the game module; without a setting the original behaviour stays, so there is nothing to undo or recalculate.
- **Built (stage 3):** game patches are listed in the project settings under "Game patches" (after the code fixes): a check mark while active, then "Apply" or "Undo", the name, and when expanded what the patch does. Both buttons rebuild the hack ROM at once (`EditorSession.SetGamePatch`). The ids are stored in `project.json` (`AppliedPatches`); new projects get every patch, older projects none (no dialog: a patch is a feature, not a fix). The builder applies a patch's instruction changes like a code fix (each checked against the original words) and tells the module through a header flag. The Features tab says where the patches are. Before an undo, `GamePatches.MissionsUsing` lists the missions that need the patch and asks; their settings stay.
- **Apply:** the editor fills the new per-mission values with the current retail behaviour, so applying alone changes nothing in the game.
- **Undo:** the original code comes back. The per-mission values stay in the missions (the game ignores them), so applying again restores the work. Before undoing, the editor lists the missions whose values differ from the retail behaviour; the point values of those missions are recalculated with the retail rules and the generated texts follow. A source that no longer works (e.g. time points at the goal ring) is reported in the point list. For the two Gyrocopter curves: the resting-position curve becomes the shared curve, with its maximum set to √(shown value).
- **Without the patch:** wherever several vehicles or missions share a value (pad radius, vehicle constants), the editor shows a short note.

5.5 Penalty messages on the HUD (built, game module)

- The HUD messages are fixed texts: `OFF_5` "5 pts. deducted" (Gyrocopter hit, code_7CF30.c, call at ROM 0x7E140) and `OFF_2` "2 pts. deducted" (Rocket Belt hit, ROM 0xAD0C4; Jumble Hopper leaving the area, ROM 0x83460). The points themselves come from the mission (COMM+0x3B4, COMM+0x414), so a changed value was counted correctly but the message kept the old number.

- If any mission's value differs from its message (GC ≠ 5, RB ≠ 2, JH ≠ 2), the three hudWarningText calls go to the module (module_penalty_hit / module_penalty_area). It shows the message, then replaces the bold digit in the HUD copy (sHudState+0xB40, codes 0x60-0x69 from the header +0xD4) with the mission's number (up to 3 digits). No message if nothing is deducted. The mission hint text RP_5_A ("deducted 2 points") is a normal text (Texts tab).
- Checked in the emulator (RB Beginner): 12 → "12 pts deducted", 0 → no message (2026-10-14). The user confirmed Gyrocopter and Jumble Hopper in ares.

6. Result sheet and texts

6.1 Rows (code data, ROM 0xD73B0, uncompressed)

`sResultPtTypes[6 vehicles][4 classes][3 tests][4]` lists the shown rows, **bottom row first**. Types: 1 Impact, 2 Accuracy, 3 Time, 4 Rings, 5 Balloons, 6 Altitude, 7 Targets, 8 Meca Hawks, 9 Hover pads, 11 Photo, 12–15 Cannonball target 1–4, 16 Formation, 0 empty. There are exactly 4 rows (4 number buffers, fixed screen positions). A source not in the table still counts towards the total, invisibly. Birdman has no entry (6 vehicles only).

6.2 Layout

- The numbers are drawn at fixed heights; rows are **bottom-aligned**. The text puts its rows at the same lines by starting with empty lines: 4 sources → 1 empty line, 3 → 2, 2 → 3, 1 → 4 (e.g. JH "Time Pts." only).
- The line above the total is **not in the text**: the game draws it as a 1-pixel box at a fixed height for every vehicle except Birdman. Fewer sources do not move it.
- "Points Deducted" row and total position depend on the vehicle (code, stays fixed): Cannonball, Hang Glider and Sky Diving have no deduction row (total one row higher).
- `_S1` (mission screen) shows "N pts." per source, the total ("100 pts.") and an optional footnote; `_S2` (result sheet) shows "/N pts." per source, "Points Deducted" and "Total". The numbers in front of the slash come from the game.

6.3 Text names and the tables in the code

- Tables (ROM offsets, US): class islands 0xD6950 (u8[7][4]), medal limits 0xD7104, result sheet names (S2) 0xD7170, mission screen names (S1) 0xD7290, result rows 0xD73B0. The name pointers are RAM addresses; ROM offset = pointer − 0x80278AD0.
- The game indexes [vehicle][class][test] without checking the test: the fourth Cannonball target reads the next class's first entry. Level 3 target 4 therefore has no S1 name (the entry is NULL).
- Sky Diving formation points come from a table in the code (D_8034FFD0 in skydiving.c, one entry per level), not from the mission (the SDFM chunk is empty).
- `<VEH>_<class><test>_S1/S2`, with VEH = HG, RP (Rocket Belt), GC; class E (no test number), A, B, P. Tables `sResultTestNames1/2` hold pointers to these names (strings around ROM 0xDCBE0).
- Shared texts: `CB_L123_S1/S2` (all Cannonball levels), `SD_L123_S1/S2` (all Sky Diving

levels), `HP_L123_S2` (all Jumble Hopper levels; S1 is per level: `HP_L1/2/3_S1`). Splitting them needs new names and changed pointers (built in stage 5, section 6.4).

- Birdman uses `BD_ALL_S1` ("free flying stage, no score") and `BD_ALL_S2` for every mission (hard-coded in resultInit, not read from the tables).

6.4 Game patch "Bonus games and Birdman" (`bonus-birdman-v1`, built in stage 5)

- **Result tables in the data area** (`ResultTables`, RAM 0x801FD100, behind the room for the 128 mission entries; data area header +0x10 = their size): S1 names u32[7][4][5] at +0x000, S2 names at +0x230, rows u8[7][4][5][4] at +0x460, the new names (ASCII) from +0x690. Every entry holds what the original code read for the same vehicle, class and test (tests 3 and 4 read the next class as before), the Birdman's names are BD_ALL_S1/S2, the rows of the project (MissionSettings.ResultRows) are written in. The original rows table is still patched too (the importer reads it).
- resultInit (the only reader of the three tables): in the index computation `x * 4 - x` becomes `x * 4 + x` (subu → addu) for vehicle and class in all three places (ROM 0xB5618/0xB5620, 0xB5650/ 0xB565C, 0xB5738/0xB573C), and the `lui` / `lw` / `lbu` of the table addresses point to the new tables (0xB563C/0xB5648, 0xB5674/0xB567C, 0xB575C/0xB57D4).
- **Texts per level:** `SD_L1_S1` … `SD_L3_S1`, `SD_L1_S2` … `SD_L3_S2`, `HP_L1_S2` … `HP_L3_S2` (9 custom texts, copies of `SD_L123_S1/S2` and `HP_L123_S2`). They exist in memory until a text is saved (EditorSession.LoadTexts adds them to the library), and the builder adds missing ones to the built text file, so the game always finds them; the importer does not count a text file that differs only by them. They take the text numbers 439–447 in a new project. Cannonball stays shared. The patch needs the code fix expanded-texts-v1. Undo: the texts stay as custom texts; the missions show the shared texts again.
- **Birdman medal limits:** gMedalPointRequirements entry 7 (ROM 0xD712E) 0/0/0 → 280/320/360. The eight calls of levelSetPointsToNextMedal (ROM 0x93C64, 0xA9114, 0xB311C, 0xB3194, 0xB31D0, 0xCE518, 0xCE530, 0xCE944) go to `module_points_to_medal`: with every medal reached the original writes "100 × tests − points" only for the main vehicles and "100 − points" for all others; for the Birdman the module writes 100 × taskGetTestCount(class, 6) − points.
- **Save file:** saveFileWrite/saveFileLoad (ROM 0x6F9FC, 0x6FA8C, delay slot 0x6FABC; 0x6FD4C, 0x6FDE4, 0x6FE14) count 3 classes only for vehicles 3–5 (`at = (veh - 3) < 3 unsigned; v0 = 4 - at`), so Birdman stage 4 is saved (Save_File_Format.md).
- Checked: VM (save and load loops write/read 61 results in the same order, Birdman classes 0–3, bonus games 0–2; the medal wrapper gives 20 to go at 380 Birdman points and leaves the other vehicles as they were) and emulator (Sky Dive 2 "Scoring" shows `SD_L2_S1`, the result sheet of Sky Dive 1 shows `SD_L1_S2` with the rows Formation, Landing Accuracy, Landing Impact).
- Not yet: Birdman with points (landing ends the mission, impact, landing limits, the code places that skip Birdman, BD texts per mission, class summary in stage 6). Birdman results unlock nothing and do not count for "all perfect" (to be decided later). The game

shows the "Level 3" banner for Birdman stage 4.

7. Goal ring (ring type 3)

- A goal ring ends the mission at once: the results come up as soon as it is passed. Retail: only RB Pilot 1 (mission 054) has one.
- The ring logic itself works for every vehicle (rings are checked against any vehicle's position). The problem is an original bug (task.c:358): instead of checking for the Rocket Belt, the code *sets* the vehicle to Rocket Belt and plays the Rocket Belt jingle. **Checked in the emulator (stage 3): a Hang Glider at a goal ring crashes the game** (the game's crash screen appears). The same bug exists for the ball target in the line next to it.
- **Built (stage 3): game patch "goal objects" (`goal-objects-v1`)**, in both blocks of taskFrameUpdate (ROM 0xCCA78 ball target, 0xCCAE0 goal ring): the store `veh = ROCKET_BELT` and the check for the Hang Glider become nops, and the call of sndSetMusic goes to `module_goal_jingle`, which plays the jingle of the vehicle being flown (HG 0x05, RB 0x09, GC 0x0D, CB 0x11, SD 0x14, JH 0x18, BM 0x1A). sndSetMusicState and func_8033FCD0 then run with the real vehicle. Checked in the emulator: Hang Glider Beginner with a goal ring ends with "GOAL!" and the Hang Glider's result sheet (ring points 15, time points 19/20). The jingle itself was not checked by ear.
- **Time points at the goal ring and the ball target (built, always, no patch button):** the module adds the time flag (0x800) when `taskGet_80346468` (D_803507A4) says that the goal ring or the ball target ended the mission. A mission without time points has an empty time curve, which gives 0. The conditions still apply. Vehicles whose flags are still 0 in the air (Hang Glider, Birdman, section 3.1) first get 0xF7F8, so their objects, altitude and distance count.

7.1 Check rings (ring type 2, shown as "type 3" in the editor)

- The score type of a ring (RNGS +0x72) is 0–3: 0 and 1 are normal rings, 3 the goal ring, and **2 an invisible check ring** (rings.c): no model (objId 0xFFFF), no radar waypoint, no sound and no HUD message when passed. Its collision still works, so flying through it activates its child rings.
- Purpose: an internal check of how far the player got. Retail: only RB Pilot 1 "Dark Cavern" (mission 054): ring 1 (active, type 2) → ring 2 (type 2) → goal ring. Before that the goal ring is inactive and shows as a grey sphere that cannot be passed.
- They are not meant to be flown through on purpose, so they are **taken out of the scoring**: their ring points are 0 in every retail mission, the Missions tab shows no points field for them (only a line "invisible check rings, no points"), and the conditions do not count them (the game module counts rings from `gRings` and skips score type 2; the editor shows "of N" without them).
- `ringsGetCleared` counts every ring with `consumedType` ≠ 0, including type 3 ("sibling of the ring taken, removed"). Retail mode 2 ("all objects") still counts check rings; in a chain this makes no difference.
- Ring struct (0x1CC bytes, `gRings` at 0x8036DA78, count u8 at 0x8036DA74): scoreType

+0x146, ringSubtype +0x147, consumedType +0x1B4 (checked in ringsGetCleared/ringsGetPoints).

- RNGS chunk (TaskRNGS, 0x84 bytes per ring): position +0x00, angles +0x0C, child count +0x1D, children +0x20, points +0x4C, size +0x54, active +0x55, score type +0x72, name +0x74.

8. Changing the source of a mission

- The sum does not care which vehicle collects what: every object type is loaded for every vehicle (`taskLoad`), and the result counts all of them.
- So changing a source is **data only**: put the objects into the mission (level editor), set the row in the result table, and generate the texts. No code change.
- What each vehicle can collect (code checks):

Source	HG	RB	GC	CB	SD	JH	BM
Curves: landing, impact, time, altitude, distance	✓	✓	✓	landing only	✓	time, altitude, distance (landing accuracy not offered since 2026-10-16: far too imprecise; no retail mission uses it)	time, altitude, distance, landing accuracy (decision revised 2026-10-16); impact is not measured
Rings (incl. goal ring with the patch)	✓	✓	✓	flag patch	✓	✓	✓
Balloons	✓	✓	✓	–	–	✓	✓
Ball target (balloon pushed in, presumably)	✓	✓	✓	–	–	✓	✓
Hover pads	–	✓	–	–	–	–	–
Targets, Meca Hawks	–	–	✓	–	–	–	–
Photos	✓	–	–	–	–	–	✓
Formations	–	–	–	–	✓	–	–
Hopper goal	–	–	–	–	–	✓ (points count with the patch, 9.1)	–

✓ for balloons = the vehicle code calls the balloon contact routine; touching a balloon pops it (fast) or pushes it (slow).

- Mode 2 ("all objects") and mode 3 count all object types together.

9. Bonus games

- **Sky Diving:** flags are always 0xFFFF, so rings and all curves (time, altitude, landing, impact) already count. Needed: the objects, the row in the result table, and separate texts per level.
- **Jumble Hopper:** the same, plus balloons and the ball target.
- **Cannonball:** flags are the constant 2 → one-instruction patch for rings. The ring points end up in the score of the target being shot at; the 4 rows are the 4 targets, so there is no own row. Whether rings stay cleared across the 3 shots needs testing. Postponed.

9.1 Jumble Hopper goal: two original bugs (fixed by the game patch "goal objects")

- **Points always 0:** the flags are passed as 16 bits, so the bits for the hopper goal (0x20000) and the ball target (0x10000) are lost in a normal run. **Built:** with the patch (module header flag 64) `module_conditions` adds both bits whenever the flags are not 0. In retail all ball targets and hopper goals have 0 points, so nothing changes for the original missions.
- **Several goals:** the loader moves the data pointer *and* uses the index, so goal 2 reads goal 3's data and so on (code_82520.c:93, func_802FB22C). **Built:** the store `sw t3, 0x28(sp)` (hopd++, ROM 0x82818, delay slot of the loop) becomes a nop. Then up to 5 goals work. (Not yet checked in the emulator: no mission has several goals before the level editor.)

10. Birdman

10.1 What already works

- The Birdman missions contain a landing curve (60 points, from development).
- The scoring function runs for Birdman; rings, balloons and photos count (flags 0xF7F8/0xF7FB after landing; 0 during the flight, see 3.1).
- The save file stores 7-bit results for stages 1–3.
- The class summary screen can show up to 4 tests (4 number rows, positions computed from the test count).
- Landing measurement: the resting position is measured like for every other vehicle (pads and runways). The impact is not measured at all (always 0).
- **Decision (revised 2026-10-16): landing stays possible for Birdman.** A Birdman mission can end through a landing, the goal ring, the hard end time or a crash; the editor will offer landing accuracy as a source for Birdman (stage 5, when Birdman gets points), so hack authors can use it if they want. Impact is not measured. The goal ring (with the patch) and the hard end time are built (stage 3).

10.2 What is needed for points

- About ten code places that skip Birdman (result sheet, number rows, line, points on the mission screen, class summary call, test summary). A checkbox per mission ("score" vs "free flight text") drives a patched check.

- A result-row table entry (the table has only 6 vehicles; it moves to a larger copy, see 11).
- `_S1/_S2` texts per mission instead of `BD_ALL_S1/S2`.
- **Medal fix (built in stage 5, section 6.4):** Birdman uses medal entry 7 (now 280/320/360) and the bonus branch, which computed "points to go" as 100 − points instead of 100 × 4 − points (now through the module). Still needed: own class summary texts with 4 test rows (today it would use the `BONUS_S3_*` texts made for 1 test; stage 6).
- The goal ring fix and time points at the goal (section 7, built).
- The mission end without landing: the retail Birdman ends only by landing or crash; goal ring (section 7) and hard end time (4.3) cover the rest (both built).

10.3 Stage 4 with points

- The save code treats Birdman as a bonus vehicle and saves only 3 classes (`IS_BONUS_VEHICLE ? 3 : 4`, in the save and load loops). Patch: 4 classes for Birdman. That shifts the photo album behind it. Old saves do not matter (no hack saves exist yet); the editor's save tab and its texts follow the new layout. **Built in stage 5** (section 6.4): the save tab shows the Birdman's points (4 stages × 4 missions) when the patch is applied.
- Effort: low (on top of 10.2).

11. Number of missions (target: 128 in total)

11.1 Limits

Limit	Where	Retail	For 128 missions
Missions loaded	loop count 0x3D in `taskInit`	61	change to the real count (one instruction)
File number of each mission	`sTaskUserFileIdxLookup`, u8 × 68	61 + 7 spare	new table with 128 entries elsewhere, pointer patched
User files (all non-kernel files)	`gUVBlockOffsets.userFiles[0x80]` in RAM	111 of 128 used	**only 17 spare (→ 78 missions)**; 128 missions need 178 user files, so the array must move to a larger RAM area (e.g. 256 entries) and its references be patched
Tests per class and vehicle	index table `[8 classes][5 tests][7 vehicles]` and the results in RAM	–	5 (fits)
Result texts and rows	`sResultTestNames1/2` `[6][4][3]`, `sResultPtTypes` `[6][4][3][4]`	3 tests, 6 vehicles	new tables `[7][4][5]` (Birdman included), pointers patched
Class summary	4 number buffers	4 tests	5 buffers (moved), texts with 5 rows
Save file	256 bytes per file: 16 bits magic, 7 bits per test, 1032 bits photos, 8 bits checksum	57 tests	128 tests use 1944 of 2032 bits (88 bits spare; the absolute maximum would be 140)
Mission settings table	data area (13.4)	–	128 entries of 0x100 bytes (built)

Note: the 128 of the user-file table and the 128 missions are a coincidence. The table counts *all* user files, not only missions.

11.2 The 111 retail user files

Numbers	Type	Count	Content
0–3	UPWL	4	level data of the four islands
4	SPTH	1	path of the ski lift
5–65	UPWT	61	missions (one file per test)
66	ADAT	1	all game texts
67–71	SPTH	5	camera paths of the title screen
72–77	3VUE	6	camera flights of the title screen
78–83	3VUE	6	camera flights of another menu screen (code_A8C30, presumably the class result screen)
84	PDAT	1	recorded flight for the credits
85–90	PDAT	6	recorded demo flights, one per pilot
91–108	PDAT	18	recorded demo flights per pilot and vehicle (Hang Glider, Rocket Belt, Gyrocopter)
109–110	SPTH	2	flight paths of the planes in the levels

The numbers are hard-coded in the code (for example demo flights 0x55–0x6C, title paths 0x43–0x4D). New missions therefore go **behind** the existing files, so that no number moves.

11.3 Notes

- The counts come from the missions, so removing tests is simple; the menus count with `taskGetTestCount`. The test selection steps through the tests, so 4–5 tests per class should need no menu change (to be tested).
- Every mission is read once at boot; 128 missions make the start slightly slower.
- The editor shows the mission count as "n / 128".
- Effort: medium (several tables move, one RAM array moves). The user-file array is the part that needs the most care, because the kernel fills it at boot.

12. Medals and class summary

12.1 Limits (ROM 0xD7104, `gMedalPointRequirements`, 8 × 3 u16)

Entry	Bronze / Silver / Gold
0 Beginner	70 / 80 / 90
1 Class A	140 / 160 / 180
2 Class B	210 / 240 / 270
3 Pilot	210 / 240 / 270
4–6 bonus games	70 / 80 / 90
7 (Birdman)	0 / 0 / 0 (280 / 320 / 360 with bonus-birdman-v1)

- License classes: **one limit per class**, shared by Hang Glider, Rocket Belt and Gyrocopter. Separate limits per vehicle need a code change in about six places plus a new table.
- Bonus games: read in at least five ways – per bonus vehicle (entry veh+1, file menu i+4), per level (entry cls+4), and fixed entry 4 (level select, also for Birdman's unlock check) or fixed CLASS_COUNT (Cannonball in the class summary). The three entries must stay equal today.
- **Per bonus vehicle and level** needs 9 entries (3 × 3) instead of 3: a new, larger table and patches at about 10–12 read sites in 6 files (scoring, class summary, level select, file menu, test summary and code_A8C30). Effort: medium–high, mostly finding and testing the read sites. Birdman's unlock check (silver in all levels of a bonus game) would then use each level's own silver limit.
- "Perfect" = 100 × number of tests (code).
- The limits also decide unlocking (silver in all three vehicles opens the next class or the bonus games), see Save_File_Format.md.

12.2 Class summary texts

- Per class: `E_S3_*`, `A_S3_*`, `B_S3_*`, `P_S3_*` with the variants BLONDS (bronze), SILVER, GOLD, PERFECT and GREAT; bonus games: `BONUS_S3_*`. They contain the limit ("< 140 pts. for Bronze Badge >") and one row per test.
- Generic: `TRY1-3` ("n tests to go"), `BLONDS/SILVER/GOLD/PERFECT/GREAT_QFY`.
- These texts are per class, not per vehicle, and are generated from the limits.

13. Memory and space

13.1 Mission objects

Limit	Retail	Note
Object memory per mission (`mem_get` pool in task.c)	5000 bytes	largest retail mission: 3260 bytes (GC Pilot 1, 24 rings); a ring takes 132 bytes
Rings	30	fixed array
Balloons	20	fixed array
Hover pads	20	fixed array
Jumble Hopper goals	5	fixed array
Map objects (radar etc.)	60 per mission	rings, pads, targets and more together
Radar targets	10	goals, ball targets, used pads and runways
Moving objects (whole game)	100	object pool (dobj)

- Planned: move the object memory to the free data area below 0x801FE000 and make it much larger (e.g. 32 KB), and enlarge the fixed arrays of the types we need (each array moves as a whole; their sizes are then raised together).
- Where the hardware gives up has to be measured: the display list holds 4200 commands per frame (the game freezes on overflow), at most 64 terrain tiles are visible per frame, collisions are limited to 32 hits. The editor should warn per mission from a measured object count on.

13.2 Island models

Everything an island needs (terrain contours UVCT, models UVMD, textures UVTX) is loaded into RAM **completely** when the island is loaded (`uvLevelAppend`). The ROM is not the limit (it can grow to 32 or 64 MB); the RAM heap is.

Island (level)	Models	Terrain contours	Textures	Unpacked total	Packed (ROM)
Holiday Island (1)	38 – 143 KB	4 – 77 KB	70 – 202 KB	424 KB	251 KB
Crescent Island (3)	54 – 160 KB	47 – 225 KB	107 – 270 KB	657 KB	423 KB
Little States (5)	103 – 362 KB	38 – 358 KB	147 – 391 KB	1112 KB	680 KB
Ever-Frost Island (10)	40 – 157 KB	11 – 209 KB	95 – 244 KB	611 KB	376 KB

(Unpacked = file data; the RAM structures are a little larger. Pilot, vehicle, HUD and objects come on top of the island.)

- Free heap without Expansion Pak: about 1.8 MB in three areas (Bug_List_Original_Code.md). Little States alone uses well over half of it.
- With an Expansion Pak the game automatically gets 4 MB more heap. **Since stage 3 every hack requires the Expansion Pak** (13.3).
- File-count limits (kernel tables, like the user files): models 363 of 400, terrain contours 101 of 128, textures 463 of 500. New islands or more detailed ones need these tables moved to larger ones too.
- Per terrain at most 1024 tiles; at most 64 visible per frame.
- The editor should show per island: size of each model/contour/texture, the island's total, and the budget (heap without/with Expansion Pak). The exact fill level can be checked in an emulator at address 0x802B892C.

13.3 Code area for new program code (built)

- RAM 0x803805E0–0x803905E0 (64 KiB, directly behind the game's variables) is reserved for the editor's program code: the program block of the memory manager ends at 0x803905E0 (uvMemInitBlocks) and the clearing of free memory at every level start begins there (uvMemClearRegions); only the upper halves of two instructions change.
- The code (the "game module", source native/game_module/module.c, compiled into the editor) lies in the last 64 KiB of the ROM, whatever its size (it moves along when the ROM grows). A loader in the code cave (uvMemScanBlocks, ROM 0x2B4D4) copies it into the code area right before app_entrypoint (call in Thread_App replaced) and calls its init function. Checked in the emulator with and without Expansion Pak; the module stays intact across level starts.
- **Since stage 3 the module is in every hack ROM built from a project**, and every hack ROM is **16 MiB** (`ProjectBuilder.HackRomSize`, RomBuildOptions.MinimumRomSize; the padding costs the BPS patch almost nothing). The project setting "allow enlarging" now means beyond 16 MiB.
- **Expansion Pak check (built):** module_init first reads osMemSize (0x80000318). With only 4 MB it draws a black screen with "EXPANSION PAK REQUIRED" / "This game needs the" / "Nintendo 64" / "Expansion Pak." (own 6×9 font, doubled) into a frame buffer at 0x80080000, hands it to the video interface (osWritebackDCacheAll, osViBlack(0), osViSwapBuffer; the video mode is already set by uvSetVideoMode) and stops. Checked in mupen64plus with 4 MB: the frame buffer in RAM holds the message and VI origin points to it; the debug port says "No Expansion Pak (4 MB of RAM): the game stops." HLE video plugins may show only black (they do not show CPU-drawn frames); real hardware and LLE emulators show it.
- First user: **debug output** (Features tab) – messages to the IS-Viewer port (ares: start with --terminal) and a ring buffer in RAM; optionally the startup steps ("Starting/Done" with time and details) and the developers' own _uvDebugPrintf messages.
- On-screen output: the module puts itself into the frame callback of `uvGfxEnd`

(gGfxCallback, otherwise used only by the snow) and draws its lines with an own 6×9 font into the finished frame; they vanish 15 s after the last message. Visible in LLE emulators (ares, Parallel Launcher); HLE plugins without frame-buffer emulation do not show CPU writes (same as the snow).

- Header flags (+0x0C): 1 startup information, 2 developers' messages, 4 debug output, 8 on screen, 16 live values, 32 Expansion Pak required, 64 game patch "goal objects". +0xE8/+0xEC: ROM offset and size of the data area image.
- The scoring hook (`module_conditions`, call at ROM 0xB2AE4) is always installed; with the debug output it reports "Scoring: flags 0x…, crash, hard end time, goal reached".

13.4 Data area for new tables (built, stage 3)

- RAM **0x801F5000–0x801FE000** (36 KiB) in front of the text tables of expanded-texts-v1: the program block of the memory manager now starts at 0x801F5000 (uvMemInitBlocks, `lui t1, 0x801F; addiu t1, t1, 0x5000`, block 5 starts at 0 as in expanded-texts-v1) and the clearing of free memory at every level start stops there (uvMemClearRegions). The change works on the retail code and on the code of expanded-texts-v1 (`DataArea.Patches`).
- Layout: header 0x100 bytes ('HHMD', version 1, number of missions, entry size 0x100), then the **mission settings table**, up to 128 entries of 0x100 bytes (0x801F5100–0x801FD100): +0 vehicle, +1 class, +2 test, +3 flags (1 crash gives 0, 2 minimum altitude, 4 minimum distance, 8 unlimited fuel, 16 hard end scored, 32 hard end gives 0, 64 time points after a crash, 128 own touchdown curve), +4 required objects per type (7 bytes), +0xC f32 minimum altitude, +0x10 f32 minimum distance, +0x14 hard end jingle, +0x15 landing flags (1 pad radius, 2 speed limits), +0x18 touchdown curve (0x54), +0x6C f32 pad radius, +0x70 f32[4] speed limits, rest reserved. Header +0x10: size of the result tables (0 = none); header +0x40: the Hang Glider's current speed limits (section 5.2a). 0x801FD100–0x801FE000: the result tables of bonus-birdman-v1 (section 6.4, 0x700 bytes; the image then reaches up to their end).
- The image lies in the ROM in the 64 KiB in front of the module; module_init copies it with _uvMediaCopy (writes back the data cache first). The table moved here from the module image (before: 160 entries of 20 bytes inside the module). Checked in the emulator: the conditions of RB Beginner are found after the level start.
- New data grows downwards from here (rule of the bug list).

14. Decisions so far (Mission tab)

- Layout: vehicle and test dropdowns at the top (game order); the second dropdown also gets the class summary/medal entries.
- Sub-tabs: **Mission Statement** (title, task, hint; a second place to edit texts, like the Texts tab), **Point Distribution** (S1, generated), **Results** (S2, generated).
- 4 point rows (the game's limit), each with a source dropdown (all sources of the vehicle + empty) and a value; the sum is shown below. The number of rows per mission can change, and the texts follow it.

- Point texts are always generated, never edited by hand. The footnote is set in the Mission tab.
- Object points are edited by hand later; the editor warns when the sum does not match. A "distribute proportionally" button may come later.
- Shared texts: Sky Diving and Jumble Hopper get separate texts per level; Cannonball stays shared for now.
- Time curve: default is plateau + linear decay; a special option switches to "as close as possible".
- Landing: the editor shows one sentence per vehicle on how the impact is measured (2.4).
- Patches, each with an apply and an undo button (handling in 5.4): points after a crash per mission (incl. the option "time points after a crash", 3.3); conditions per object type for time points (4.4, built through the module without a button); good landing per mission; two separate Gyrocopter landing curves; **goal objects (built)**.
- Hard end time (built): off by default; a choice per mission, "the flight is scored" (default) or "0 points"; no patch button. The footnote says when the mission ends.
- Goal ring: usable for every vehicle (game patch "goal objects", one patch together with the ball target and the hopper goals) and always gives time points (0 without a time curve; no patch button, also for the ball target).
- New projects get every game patch automatically; older projects are not asked.
- Jumble Hopper: balloons instead of rings are fine; both goal bugs are fixed by the patch.
- Birdman: points switchable per checkbox (text vs. points); medal calculation fixed; stage 4 with points; landing accuracy offered as a source (revised 2026-10-16: the user wants to leave the choice to the hack authors); a mission can also end through goal ring, hard end time or crash.
- Missions: up to 128, results tables for up to 5 tests per class, class summary for 5 tests; the editor shows "n / 128".
- Memory: larger object memory and arrays per mission; the editor shows the space each island model needs and the space available (13).
- Islands: one island per vehicle and class; the editor writes it to every mission of the class (COMM byte 3) and to the class menu table (0xD6950) together.
- Hardware (built): every hack requires the Expansion Pak (8 MB RAM; without it a black screen with "EXPANSION PAK REQUIRED"), and every hack ROM is 16 MiB.
- Graphics: the display-list limit (4200 commands per frame) is fixed; the editor warns in time.
- Islands: replacing the four island models is planned (with room for larger models); more than four islands later.
- Memory extension (later): keep room for **additional classes and bonus game levels**, not only more missions per class (index table, result tables, class menus, save layout, medal table).
- Photo missions: new photo subjects may come; plan room for up to **10 photo curves** per mission (today 6 slots in COMM, chosen by fixed code per subject – new subjects need

code and a larger table).

- Stage 2 (done): points, rows, ring points, time curve (plateau or target time), "Land now", conditions, deductions, reaction threshold and footnote are edited in the Missions tab; the point texts are generated (TextPoints for object rows, MissionSettings in project.json, result rows patched into the code table at build). After stage 2: rows can be reordered by dragging; time and altitude curves can be edited point by point (up to 10 points, steps allowed); a new altitude row gets a curve that reaches full points at altitude 500; the point texts (S1/S2) are locked in the Text tab.
- Stage 3 (Step 32): 16 MiB, Expansion Pak check, module always built in, data area with the mission settings table, game patch system with "goal objects", goal time points, hard end time per mission.
- Later: Cannonball rings; medal limits per bonus vehicle and level.
- "Points Deducted" stays as it is.
- Gyrocopter square-root conversion stays invisible.
- Stage 5 (Step 35): game patch "Bonus games and Birdman" (result tables, texts per level, Birdman medal limits and stage 4 in the save file, Birdman points in the save tab); Birdman with points follows separately.
- Stage 4 (Step 34): no patch buttons; three crash choices (with a hint for early crashes and a footnote line), the Gyrocopter's own touchdown curve in percent, the good landing values of the vehicle shown, speeds in km/h like the HUD, runway widths in the Missions tab (later also in the level editor).